

Microservices API Framework

Opinionated Go framework for building observable microservices with tracing, circuit-breaking, and gRPC/REST dual-mode routing.

Go	gRPC	Prometheus	Kubernetes
----	------	------------	------------

VERSION	STATUS	LICENSE	GO	MODULE	UPDATED
1.7.3	Open Source	Apache-2.0	1.23+	go.acme.dev/svc	Sep 2026

1. Overview

This framework gives Go services a single opinionated path from a protobuf definition to a running, observable process. One service definition is served simultaneously over gRPC and REST — the REST surface is generated from the same descriptors rather than hand-written — so internal callers get streaming and strong typing while external callers and browsers get ordinary JSON over HTTP/1.1.

The opinions are deliberate. Tracing, metrics, structured logging, deadline propagation, circuit breaking, and graceful shutdown are on by default and configured through one struct; there is no arrangement of the framework in which a handler runs without a trace context or a request histogram. Teams get consistent telemetry across hundreds of services without each one re-deriving a middleware stack.

2. Request lifecycle

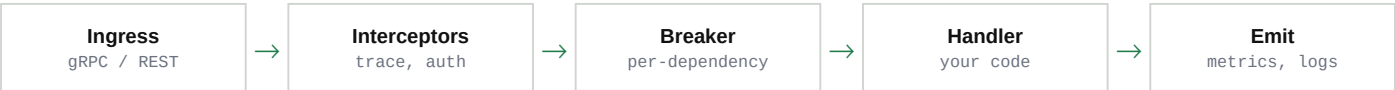


Figure 1 — Both protocols converge on one interceptor chain, so a REST call and a gRPC call to the same method produce identical telemetry.

Deadlines propagate end to end. An inbound deadline becomes the context budget for every downstream call, minus a configurable hedge, so a slow dependency cannot consume budget that the caller has already abandoned. When the budget is exhausted the framework fails fast with DEADLINE_EXCEEDED rather than issuing a call that cannot finish in time.

3. Interceptor stack

Recovery	Converts panics into INTERNAL with a stack trace on the error log, never a dropped connection.
Telemetry	OpenTelemetry span per RPC, RED metrics, and a request-scoped structured logger.
Auth	mTLS peer identity or JWT validation; populates a typed principal on the context.
Governor	Per-method concurrency limits, token-bucket rate limiting, and load shedding under queue pressure.
Breaker	Sliding-window circuit breaker keyed by downstream dependency, with half-open probes.
Validation	Protobuf field constraints enforced before the handler sees the message.

4. Resilience defaults

CONTROL	DEFAULT	RATIONALE
Retry	3 attempts, jittered backoff	Idempotent methods only, marked in the proto option.
Circuit breaker	50% errors / 20 req window	Half-open probe every 5 s; trips per dependency, not globally.
Timeout	Inherited, 250 ms hedge	Downstream budget always shorter than the inbound deadline.
Concurrency	256 in-flight per method	Bounded queue; overflow sheds with RESOURCE_EXHAUSTED.

5. Service definition

```
srv := svc.New(svc.Config{
    Name:      "checkout",
    GRPCPort:  9090,
    HTTPPort:  8080,           // REST gateway, generated from descriptors
    Telemetry: svc.Telemetry{
        OTLPEndpoint: "otel-collector:4317",
        SampleRatio:  0.05, // tail sampling keeps all errors
    },
})

// Downstream clients inherit breakers, deadlines and tracing.
inv := srv.Client("inventory", svc.Breaker{ErrorRatio: 0.5, Window: 20})

pb.RegisterCheckoutServer(srv, &handler{inventory: inv})
srv.Health(func(ctx context.Context) error { return db.PingContext(ctx) })

log.Fatal(srv.Run(ctx)) // blocks; drains on SIGTERM
```

Listing 1 — A complete service. Ports, telemetry, health, and shutdown are wired by the framework.

6. Observability contract

- **Metrics.** Every method exports request count, error count by code, and a duration histogram under a stable naming scheme, plus breaker state and in-flight gauges. Dashboards and alert rules are therefore portable across services without edits.
- **Traces.** W3C trace context is propagated over both protocols. Spans carry the method, peer identity, and retry attempt number, so a retry storm is visible as sibling spans rather than a single slow call.
- **Logs.** The request-scoped logger emits JSON with trace and span IDs attached, which makes log-to-trace navigation a direct lookup rather than a timestamp search.

7. Benchmarks

Measured on 4 vCPU pods, 1 KiB messages, full interceptor stack enabled, against a no-op handler. Overhead is the cost the framework adds on top of raw transport.

SCENARIO	THROUGHPUT	P99 LATENCY	FRAMEWORK OVERHEAD
gRPC unary	48k req/s	4.1 ms	0.18 ms
gRPC streaming	112k msg/s	2.3 ms	0.09 ms
REST gateway	21k req/s	9.6 ms	0.74 ms

8. Running on Kubernetes

- Liveness and readiness are distinct: readiness fails during drain so the endpoint is removed before the process stops accepting work, while liveness stays green to avoid a kill mid-drain.
- SIGTERM starts a bounded drain — refuse new streams, finish in-flight RPCs, flush spans and metrics — that must complete within `terminationGracePeriodSeconds`; the default 25 s drain assumes 30 s.
- Service discovery is plain headless DNS with client-side round robin. There is no sidecar requirement, and the framework coexists with a service mesh by disabling its own mTLS layer.

9. Roadmap

RELEASE	PLANNED WORK	TARGET
1.8	Adaptive concurrency limits derived from observed latency gradient.	Q4 2026
1.9	Native gRPC-Web support without a proxy hop.	Q1 2027
2.0	Pluggable transport interface; module split to trim the dependency tree.	Q2 2027